

Lab 3 Report by Alex Cooper

This lab revolves around the use of PEfile, a tool that can be used to analyze files and detect instances of malware characteristics. Unlike other labs that are completed exclusively in the Windows XP environment, this analysis was performed on my local machine. This will show in the screenshots of my Windows Subsystem for Linux (WSL) terminal but functionally the analysis works the same across systems. I used WSL because the lab3 instructions mention wget, which is a Linux command.

To begin with the lab steps 1-4 are set up instructions. Since I used my local machine, I did not SSH into badger CTF (I do not believe this is possible anymore). From there I completed steps 2 and 3 by downloading the required files from D2L, the zip file and the enum_export.py. For step 4 I used the File app on my Windows laptop to extract all the files from the zip.

From there I ran the command provided in step 5 and got the following results:

```
alex_cooper@Coops-Laptop:/mnt/c/Users/coope/downloads$ python3 enum_export.py ./Lab3/malware_lab_1/16d6b0e2c77da2776a88dd88c7cfc672
0x100011e0      b'CreateDatabaseQueryObject'      1
0x100011e0      b'DataImporterMain'              2
0x100011e0      b'FlashboxMain'                 3
0x100010d0      b'KugouMain'                    4
alex_cooper@Coops-Laptop:/mnt/c/Users/coope/downloads$ |
```

For step 6 we analyzed the above output as well as the python code. The output shows the enum_export.py provides an address, a byte string, and an ordinal of the function. Using nano, I read the Python code:

```
GNU nano 6.2                                enum_export.py
import pefile
import sys

malware_file = sys.argv[1]
pe = pefile.PE(malware_file)

if hasattr(pe, 'DIRECTORY_ENTRY_EXPORT'):
    for exp in pe.DIRECTORY_ENTRY_EXPORT.symbols:
        print("%s \t %s \t %s" % (hex(exp.address + pe.OPTIONAL_HEADER.ImageBase), exp.name, exp.ordinal))
```

The code has two imports, one of which is the PEfile library (I did have to install this library), the other is the sys library. From there, the code reads in an input from the command line and assigns a variable to the PEfile object of the command line file. Then, it checks that the PEfile is valid and loops through the file looking for the address, name, and ordinal. The code takes those results and prints them to the command line.

Step 7 requires the rewriting of the above code implementing the functionalities of the enum_export.py while also allowing the program to scan a folder instead of a single file and analyze the file structure for two rules. One rule is if 3 or more exports share an address it is malware. The second rule is if 3 or more exports share the same offset it is malware. The program will then identify if a file is malware and what rule it violated. The generated code is below:

```

GNU nano 6.2 lab3.py
import pefile
import os
import sys
from collections import Counter

#Initialize
directory = sys.argv[1]

for filename in os.listdir(directory):
    filepath = os.path.join(directory, filename)

    if not os.path.isfile(filepath):
        continue

    print("\nFilename:", filename)
    print("-----")

    try:
        pe = pefile.PE(filepath)

        if hasattr(pe, 'DIRECTORY_ENTRY_EXPORT'):
            exports = pe.DIRECTORY_ENTRY_EXPORT.symbols
            addresses = []

            for exp in exports:
                name = exp.name.decode() if exp.name else "None"
                address = exp.address + pe.OPTIONAL_HEADER.ImageBase
                addresses.append(address)

                print(hex(address), "\t", name, "\t", exp.ordinal)

            print("Total exports:", len(exports))

            address_counts = Counter(addresses)

            malware_flag = False
            #Check for "more than 3 of same address"
            for addr, count in address_counts.items():
                if count >= 3:
                    print("Malware detected: 3+ exports share address", hex(addr))
                    malware_flag = True

            offsets = []

            for i in range(len(addresses)-1):
                offset = addresses[i+1] - addresses[i]
                offsets.append(offset)

            offset_counts = Counter(offsets)
            #Check for "more than 3 of the same offset"
            for off, count in offset_counts.items():
                if count >= 3:
                    print("Malware detected: 3+ exports share offset", hex(off))
                    malware_flag = True

            if not malware_flag:
                print("No malware pattern detected.")

            else:
                print("No export table found.")

    except:
        print("Invalid PE file.")

```

The above code functions as outlined by the lab3 instructions. It first reads a directory in from the command line and uses os library to get all the files in that directory. It then utilizes the initial functionality from enum_export.py to get the address, name and ordinal from each file using a loop to process each file. The code then checks for 3 or more exports using the same address and updates the malware_flag variable if this occurs. The offset is then checked using similar logic to the first rule check. The output for each file is printed on the terminal and includes the file being checked, its exports addresses, names, and ordinals, a designation, and then what rules it violated and where. See the below example output:

```
Filename: 16d6b0e2c77da2776a88dd88c7cfc672
-----
0x100011e0      CreateDatabaseQueryObject      1
0x100011e0      DataImporterMain                2
0x100011e0      FlashboxMain                  3
0x100010d0      KugouMain                    4
Total exports: 4
Malware detected: 3+ exports share address 0x100011e0

Filename: 1c1131112db91382b9d8b46115045097
-----
0x100014a0      AfxGetHttpRequestMgr          3
0x100014b0      AfxGetHttpRequestMgr          4
0x10001490      InitInstance                  2
0x10001490      MessageLoop                   1
Total exports: 4
No malware pattern detected.
```

For the writing of this lab report no AI was used outside of general tools for spell-checking and grammar correction. All the screenshots and content are human generated. AI, specifically ChatGPT, was used for code generation alongside using the code provided in enum_export.py. AI was specifically used to create logic that checks for rule violation.